OPERATING SYSTEMS: ORANGE LEVEL PROBLEM 2 REPORT

NAME: NANDHANA S

SRN: PES1UG24AM440

SECTION: H

PHASE 1:

1A)

```
make: *** [Makefile:14: object.o] Error 1
nandhana@Nandhana:/mnt/c/Users/nandh/PES1UG24AM440-pes-vcs$ make test_objects
./test_objects
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function 'object_write':
object.c:142:45: warning: '/.tmpXXXXXX' directive output may be truncated writing 11 bytes into a region of size between 1 and 512 [-Wformat-truncation=]
  142 |     snprintf(tmp_path, sizeof(tmp_path), "%s/.tmpXXXXXX", dir);
      |                                             ^~~~~~~~~~~
In file included from /usr/include/stdio.h:980,
                 from pes.h:8,
                 from object.c:11:
In function 'snprintf',
    inlined from 'object_write' at object.c:142:5:
/usr/include/x86_64-linux-gnu/bits/stdio2.h:54:10: note: '__builtin___snprintf_chk' output between 12 and 523 bytes into a destination of size 512
   54 |   return __builtin___snprintf_chk (__s, __n, __USE_FORTIFY_LEVEL - 1,
      |          ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
   55 |                                    __glibc_objsize (__s), __fmt,
      |                                    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
   56 |                                    __va_arg_pack ());
      |                                    ~~~~~~~~~~~~~~~~~
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1B)

```
nandhana@Nandhana:/mnt/c/Users/nandh/PES1UG24AM440-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
nandhana@Nandhana:/mnt/c/Users/nandh/PES1UG24AM440-pes-vcs$
```

PHASE 2:

2A)

```
nandhana@Nandhana:/mnt/c/Users/nandh/PES1UG24AM440-pes-vcs$ make clean
make test_tree
./test_tree
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function 'object_write':
object.c:142:45: warning: '/.tmpXXXXXX' directive output may be truncated writing 11 bytes into a region of size between 1 and 512 [-Wformat-truncation=]
  142 |     snprintf(tmp_path, sizeof(tmp_path), "%s/.tmpXXXXXX", dir);
      |                                             ^~~~~~~~~~~
In file included from /usr/include/stdio.h:980,
                 from pes.h:8,
                 from object.c:11:
In function 'snprintf',
    inlined from 'object_write' at object.c:142:5:
/usr/include/x86_64-linux-gnu/bits/stdio2.h:54:10: note: '__builtin___snprintf_chk' output between 12 and 523 bytes into a destination of size 512
   54 |   return __builtin___snprintf_chk (__s, __n, __USE_FORTIFY_LEVEL - 1,
      |          ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
   55 |                                    __glibc_objsize (__s), __fmt,
      |                                    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
   56 |                                    __va_arg_pack ());
      |                                    ~~~~~~~~~~~~~~~~~
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2B)

```
nandhana@Nandhana:/mnt/c/Users/nandh/PES1UG24AM440-pes-vcs$ make clean
make
./pes init
echo "hello" > test.txt
./pes add test.txt
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Initialized empty PES repository in .pes/
Adding: test.txt
nandhana@Nandhana:/mnt/c/Users/nandh/PES1UG24AM440-pes-vcs$ find .pes/objects -type f
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
nandhana@Nandhana:/mnt/c/Users/nandh/PES1UG24AM440-pes-vcs$ xxd .pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 | head -20
00000000: 626c 6f62 2036 0068 656c 6c6f 0a         blob 6.hello.
nandhana@Nandhana:/mnt/c/Users/nandh/PES1UG24AM440-pes-vcs$ cat .pes/index
100777 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776269517 6 test.txt
nandhana@Nandhana:/mnt/c/Users/nandh/PES1UG24AM440-pes-vcs$
```

PHASE 3:

3A)

```
nandhana@Nandhana:/mnt/c/Users/nandh/PES1UG24AM440-pes-vcs$ make pes
make: 'pes' is up to date.
nandhana@Nandhana:/mnt/c/Users/nandh/PES1UG24AM440-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
nandhana@Nandhana:/mnt/c/Users/nandh/PES1UG24AM440-pes-vcs$ echo "hello" > file1.txt
nandhana@Nandhana:/mnt/c/Users/nandh/PES1UG24AM440-pes-vcs$ echo "world" > file2.txt
nandhana@Nandhana:/mnt/c/Users/nandh/PES1UG24AM440-pes-vcs$ ./pes add file1.txt file2.txt
Adding: file1.txt
Adding: file2.txt
nandhana@Nandhana:/mnt/c/Users/nandh/PES1UG24AM440-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  .gitignore
  untracked:  commit.c
  untracked:  commit.h
  untracked:  index.c
  untracked:  index.h
  untracked:  Makefile
  untracked:  object.c
  untracked:  pes.c
  untracked:  pes.h
  untracked:  README.md
  untracked:  test.txt
  untracked:  test_objects
  untracked:  test_objects.c
  untracked:  test_sequence.sh
  untracked:  test_tree
  untracked:  test_tree.c
  untracked:  tree.c
  untracked:  tree.h
```

3B)

```
  untracked:  tree.h

nandhana@Nandhana:/mnt/c/Users/nandh/PES1UG24AM440-pes-vcs$ cat .pes/index
100777 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776270233 6 file1.txt
100777 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776270238 6 file2.txt
nandhana@Nandhana:/mnt/c/Users/nandh/PES1UG24AM440-pes-vcs$
```

PHASE 4:

4A)

```
nandhana@Nandhana:/mnt/c/Users/nandh/PES1UG24AM440-pes-vcs$ ./pes init
echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"

echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"

./pes log
Initialized empty PES repository in .pes/
Adding: hello.txt
Committed: 201b8d641fa7... Initial commit
Adding: hello.txt
Committed: c4fb6396eb08... Add world
Adding: bye.txt
Committed: 07a00551cb89... Add farewell
commit 07a00551cb89bf9b5e8abf7b583d6fe155c98aa1ee51b1a0c0f30b4e9413934a
Author: PES User <pes@localhost>
Date:   1776270575

    Add farewell

commit c4fb6396eb085862f49b81bd0d7fa3376a3abc13d1f335f36d0a0593b4ef1aa2
Author: PES User <pes@localhost>
Date:   1776270575

    Add worldell

commit 201b8d641fa79c511ea178c03a06af3458658bb74f7eeb5400a68b6ec5821f9b
Author: PES User <pes@localhost>
Date:   1776270574

    Initial committial commiQ
```

4B)

```
nandhana@Nandhana:/mnt/c/Users/nandh/PES1UG24AM440-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/03/6d68ddc70e370c469230cfbc3b179646f63f5ed43eaa07e210d9e048d1e3d5
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/07/a00551cb89bf9b5e8abf7b583d6fe155c98aa1ee51b1a0c0f30b4e9413934a
.pes/objects/15/c3182060db81e6e37329382a028c906276dd30b139806d4729b7eb16e90a29
.pes/objects/20/1b8d641fa79c511ea178c03a06af3458658bb74f7eeb5400a68b6ec5821f9b
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/30/b91bee094fe0a1a7c4468def0a7a11d1e08c691941ae7d04e6c772000ffb49
.pes/objects/55/035b642eb570f86affb01cf48a7af4ef94820f45f1f39fae7397410b77d221
.pes/objects/5e/edc0d684769e46ee368d7f216cceaa043556efba866b6fcef0847c719b2c3e
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/78/65d299bacad887ab44eb9e5d9481efb9f1fc5ebbcbe8a62cd03c0e5196c7d9
.pes/objects/84/151b61e42d03910ff9f820eb31174ad5828fd5a783f943769819b1847c10b4
.pes/objects/a6/0398b4fb5dbfba436a49b2855e7abaac5aa38006e0e1806692fd01a3a68076
.pes/objects/c4/fb6396eb085862f49b81bd0d7fa3376a3abc13d1f335f36d0a0593b4ef1aa2
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/e7/9288e8b53bba03d17f88c3443484c38e21de4db4d9ad5f467c4de876ce7be9
.pes/objects/fd/1eafa6a03f48d6a5f00a2630f462f1730acc82f545d8cda1176e7cea8c96cf
.pes/refs/heads/main
```

4C)

```
nandhana@Nandhana:/mnt/c/Users/nandh/PES1UG24AM440-pes-vcs$ cat .pes/refs/heads/main
7865d299bacad887ab44eb9e5d9481efb9f1fc5ebbcbe8a62cd03c0e5196c7d9
nandhana@Nandhana:/mnt/c/Users/nandh/PES1UG24AM440-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Full integration test:

```
nandhana@Nandhana:/mnt/c/Users/nandh/PES1UG24AM440-pes-vcs$ make test-integration
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Adding: file.txt
Adding: hello.txt
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: ad47f376b0b5... Initial commit

Log after first commit:
No commits yet.

--- Second Commit ---
Adding: file.txt
Committed: d7d6aa024e3f... Update file.txt
```

```
Committed: ad47f376b0b5... Initial commit

Log after first commit:
No commits yet.

--- Second Commit ---
Adding: file.txt
Committed: d7d6aa024e3f... Update file.txt

--- Third Commit ---
Adding: bye.txt
Committed: 46f9a895a173... Add farewell

--- Full History ---
No commits yet.

--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
46f9a895a17394e487543d142598f03f3d11bb16490bc0c5998da8f14e3c4222

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/46/f9a895a17394e487543d142598f03f3d11bb16490bc0c5998da8f14e3c4222
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/ad/47f376b0b5f0536fd37748993a7a9e189b3526cd25ef92ed1c9545ffa9b572
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/d7/d6aa024e3f23e5b013515df1e08a25c738cb8931ec3a8cb0072b9e0e41ee93
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

Phase 5 & 6: Analysis-Only Questions

Branching and Checkout

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

Ans. To implement pes checkout <branch>, we first update the repository metadata inside .pes/. Since a branch is just a file in .pes/refs/heads/ containing a commit hash, checkout begins by reading the target branch file and updating .pes/HEAD to point to that branch reference (not a raw commit hash). This ensures future commits continue from the correct branch tip. Next, we retrieve the commit pointed to by the branch, extract its tree object, and reconstruct the working directory to match that tree. This involves writing all tracked files from blob objects, creating directories as needed, and deleting files in the working directory that do not exist in the target tree. Finally, the index (staging area) must also be replaced with the snapshot of the target commit's tree so that staging state matches the checked-out version.

This operation is complex because it is not just a pointer switch but a full filesystem transformation. The system must safely reconcile differences between the current working directory and the target branch without losing user data. It must handle file creation, deletion, and modification in a consistent way while ensuring the index, HEAD, and working directory remain synchronized. Additionally, care must be taken not to overwrite uncommitted user changes, which requires careful diffing between states.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

Ans. A file is modified locally AND also differs in target branch → unsafe overwrite

Using only index + object store: For each file tracked in index:

1. Get current working directory version : compute hash of file content → cwd_blob
2. Get index version : index_blob
3. Get target branch version : commit → tree → blob → target_blob

Conflict condition: A file is dirty and conflicting if:

cwd_blob != index_blob (user modified locally)
AND
index_blob != target_blob (branch changes would overwrite it)

Action: Abort checkout and Report list of conflicting files

The index is the last known safe state.
You only allow checkout if local changes do NOT collide with branch changes.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Ans. Detached HEAD means: HEAD → commit hash directly (not a branch).

When you commit, a new commit is created. Its parent = current HEAD commit but no branch reference moves forward. So the commit becomes unreachable unless saved

Graphically:

```
A → B → C   (HEAD detached at C)
      \
       D   (new commit)
```

D is not on any branch.

To recover commits: If you know commit hash:

pes checkout <commit-hash>
pes branch recovery <commit-hash>

Or via reflog (if implemented): HEAD history stores previous positions. You can find lost commit and branch from it.

Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Ans. Garbage collection in a Git-like system works by identifying all reachable objects starting from known roots, such as branch tips in .pes/refs/heads/ and the current HEAD. From these roots, we traverse the object graph recursively, following commit-to-tree and tree-to-blob relationships, marking every visited object as reachable. A hash set is used to efficiently track visited objects and avoid repeated processing.

After marking, we perform a sweep phase where every object in the object store is checked. Any object not marked as reachable is considered garbage and can be safely deleted. Given a repository with 100,000 commits and 50 branches, the traversal would typically involve visiting all commits, their associated trees, and all blobs referenced across history. This can easily amount to millions of objects, so the algorithm operates in linear time relative to reachable objects rather than total storage size.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Ans. Running garbage collection concurrently with a commit operation can lead to race conditions where objects are incorrectly deleted. This happens because GC first determines reachable objects at a snapshot in time, but during this process, a new commit may be created that references newly written objects which GC has not yet marked. If GC finishes its sweep before these objects are marked, it may incorrectly classify them as unreachable and delete them, even though they are required by the in-progress commit.

Real Git avoids this issue through locking and safety mechanisms. It uses repository locks during critical write operations to prevent GC from running simultaneously with object creation. Additionally, objects are written atomically and only considered for deletion after a grace period, ensuring that recently created or unreferenced objects are not immediately removed. This prevents race conditions and guarantees repository integrity.